

“Image_Censorship_CUDA” Project Report

Project developers and report authors:

- Cocilova Marco
- Ramos Abigail

Project reference:

https://github.com/Marclova/Image_censorship_CUDA.git

Published as academic report for University exam by the University of Camerino.

Document release date:
August 2026

INDEX

- 1- Introduction
 - 1.1- Project Pipeline and Structure
 - 1.2- Usage instructions
- 2- Project Configuration
 - 2.1- CLion configuration
 - 2.2- VSCode configuration
- 3- CPU Implementation (C++)
 - 3.1- User Interface
 - 3.2- Data Model
 - 3.3- Image Loader
 - 3.4- Process Manager
- 4- GPU Implementation (CUDA)
 - 4.1- Implemented Algorithm
 - 4.2- Kernel Data Models
 - 4.3- Threads Organization
- 5- Conclusions and Results
 - 5.1- Possible Improvements

1- Introduction

The project was born with the objective to realize a parallelized solution for the elaboration of digital images, specifically implementing the blurring feature.

The applicative works by using a combination of C++ and CUDA code, having the C++ code as pipeline administrator and the CUDA implementation to fulfill the blurring feature.

As widely explained in section 2, the project configuration required to match the versioning of the two used compilers, which are **MSVC** and **NVCC** respectively needed for C++ and CUDA. Moreover non irrelevant obstacles have been met from the compatibility between the selected GPU (NVIDIA) and the devices used for the code development. More precisely one of the devices has resulted to be too old to support the latest versions of the NVIDIA software, while the other was using a different GPU entirely.

1.1- Project Pipeline and Structure

The applicative in its current state has as one and only feature the manual selection of rectangular areas for blur application.

The project, although having been entirely developed with a function-oriented approach, makes extensive use of data models (implemented as struct in specific files) and interfaces (implemented as header files) for easier communication between modules and within components of the same module.

Furthermore a layered structure is enforced, splitting the implementation into different levels of abstraction; going from the “UI” module as the most abstract, down to the “CPU” and “GPU” modules to work with more low-level and flattened data structures.

The general pipeline consists in a direct flow through all the implemented modules, leaving to the user just a few choices about settings and area selection.

The main process flow may be described by the following steps:

1. The application starts with the main calling the 'start_ui()' function of the UI module.
2. The UI module asks the users to choose the image to blur, the areas of the image to blur and the intensity of the blurring.
3. The UI creates a mask for each selected area and sends all the data by requesting a blurring operation to the CPU through a function call to the interface.
4. The CPU, composed of a process manager and multiple internal services, makes a first general conversion and initializations are performed onto the received data before propagating the request to the GPU.
5. The GPU, divided within an “host” API programmed in C and a “device” actuator programmed in CUDA, makes the last case-specific adaptations of the data before calculations are performed and the result is returned.

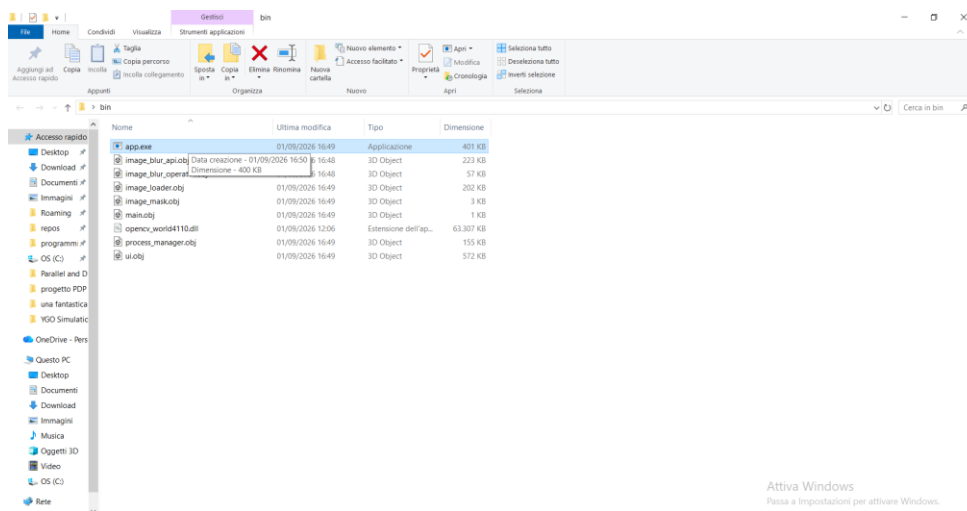
- The result is propagated back to the UI module, where it's converted into a png image and saved into the same folder of the input image. The name of the generated output image is the same as the input one plus the attached “_blurred” digit at the end.

Further details about used technologies and implementations are provided in **sections 3 and 4**.

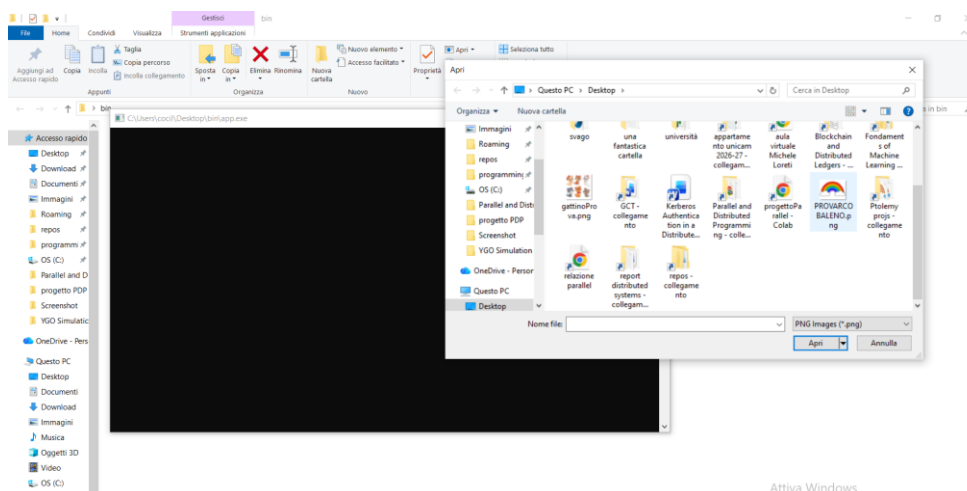
1.2- Usage Instructions

To run the application, follow these steps:

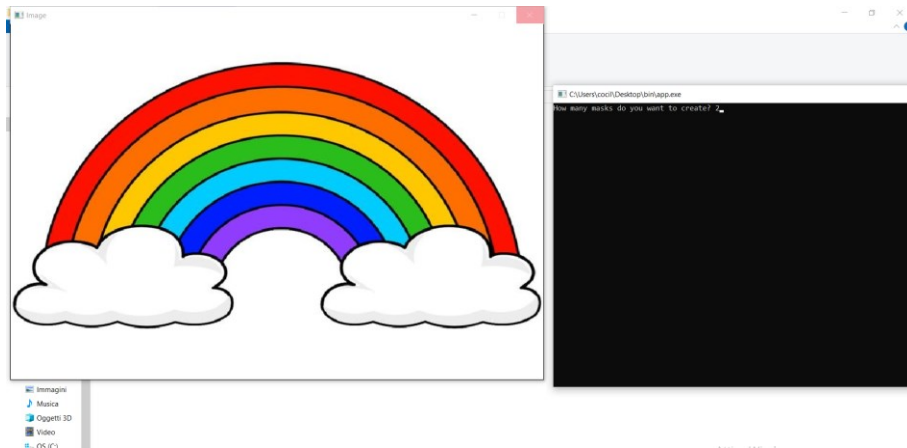
- Open the project and select “app.exe,” as shown in the screenshot.



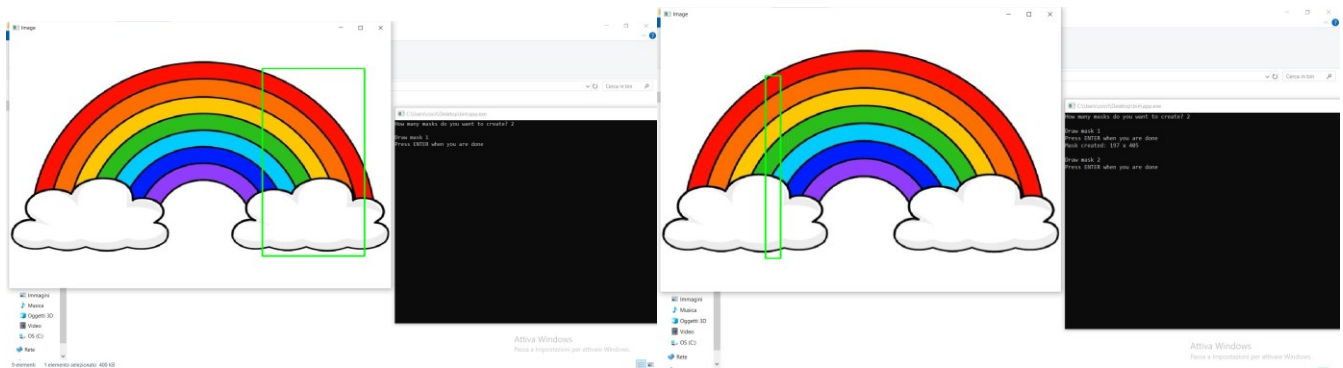
- Immediately afterward, it will ask you which image you want to select to edit; select an image with the .png file extension.



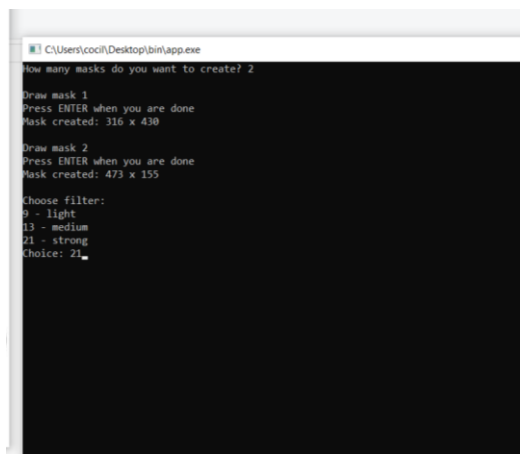
3. Then it asks you how many masks you want to create in the image; masks are the areas of the image that you want to blur.



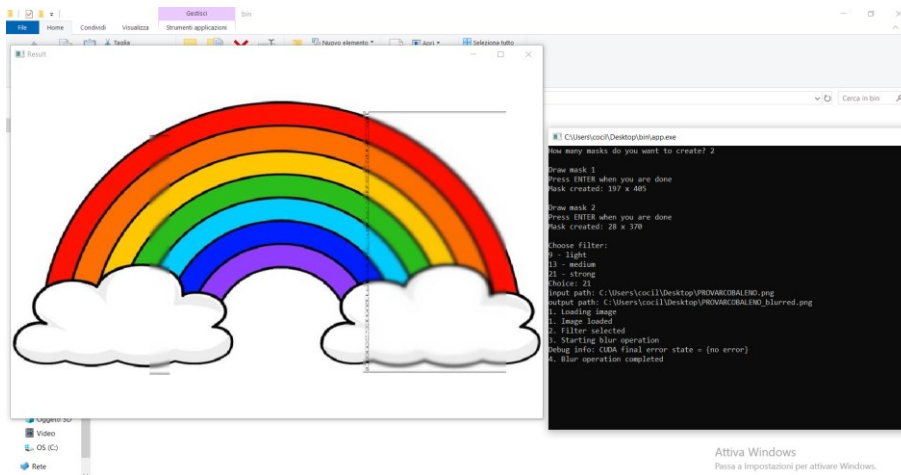
4. Start creating your first mask by using the mouse to drag and select the desired area, then press Enter when you're done and create the second mask (in this example, we've chosen to create two masks) the same way as the first one, and press Enter at the end each time.



5. Once you've created all the masks, it will ask you what type of filter you want to use for all of them; press Enter once you've made your selection.



6. It will generate the blurred image and display another window called “Result.”



2- Project Configuration

The project environment must be configured to allow the C++ and CUDA source code to be compiled and linked into a single executable.

The project was initially configured independently on two different development devices. Since the two machines were equipped with different GPUs and therefore supported different software and driver configurations, only one of the two environments could be extended to support CUDA development.

Consequently, the configuration for VSCode was selected as the reference development environment.

Despite only one of the two configurations, further explored in the following paragraphs, has been completed to execute the whole project, the Clion configuration has been satisfactory configured to compute the C++ (covering the 'UI' and 'CPU' modules) for development and testing independently from the GPU module.

2.1- CLion configuration

To develop the Parallel project, it was necessary to create a complete C++ development environment on Windows. The environment was set up using JetBrains CLion, specifically version CLion 2026.1.1.

To develop the Parallel project, it was necessary to create a complete C++ development environment on Windows. The environment was set up using JetBrains CLion, specifically version CLion 2026.1.1.

In particular, CLion allows you to:

- organize .cpp and .h files;
- write and edit code;
- display errors generated during compilation;
- configure CMake;
- configure the toolchain;
- start compilation;
- run the program;
- use the debugger.

To enable compilation of the C++ project on Windows, the C++ development tools provided by Visual Studio were installed, including the MSVC (Microsoft Visual C++) compiler, Microsoft's C++ compiler.

The executable that physically carries out the C++ code compilation is `cl.exe`.

Following installation, the CLion Toolchain was configured by specifying the necessary tools for building the project, including MSVC for compiling the C++ code, CMake for project configuration, and Ninja as the build system.

CMake was used for project configuration and management.

CMake is a build configuration system: the project's features and how it should be built are defined through the CMakeLists.txt file.

For example, the CMakeLists.txt specifies:

- the project name;
- the C++ standard used;
- the source files;
- the header directories;
- the required external libraries;
- how the libraries should be linked.

In the Parallel project, CMake was also used to locate and link OpenCV, which was subsequently installed via vcpkg.

Ninja, a build system particularly suited for C/C++ projects, was used to perform the actual build execution.

Furthermore, Ninja avoids full project re-compilation whenever possible, re-executing only the operations affected by changes made to the code.

2.2- VSCode Configuration

This configuration was chosen to represent the project, as it is the only one that supports the compilation of both the C++ code, through the **MSVC** compiler, and the CUDA code, through the **NVCC** compiler.

It is designed to support **NVIDIA GPUs** and is defined through a set of JSON configuration files for Visual Studio Code, including '[tasks.json](#)', '[launch.json](#)', and '[c_cpp_properties.json](#)'. These configuration files are available in the GitHub repository referenced on the first page of this document.

The versioning of the components explained below have been chosen to guarantee compatibility with the relatively old device used both for testing and development.

The following paragraphs cover a step-by-step guide of the performed configuration and an explication of the versions chosen for the downloaded drivers and libraries.

The VSCode launch configuration is available on the GitHub repository referenced in the first page of this document.

2.2.1- Step-by-step Guide

1. Download from the internet the 'OpenCV' library and the NVCC compiler.
2. Define the tasks.json to make the MSVC and NVCC compilers collaborate:
 - **Build task:** coordinates the compilation and linking tasks required to generate the executable.
 - **Setup task:** creates the required directories and copies the necessary DLL files to the output directory.
 - **Compile CUDA task:** invokes NVCC to compile the .cu source files.
 - **Compile C++ task:** invokes MSVC to compile the .cpp source files.
 - **Linker task:** links the object files generated during compilation with the required external libraries to produce the final executable.
3. Define the following references to the downloaded library in the project configurations (The following paths refer to the OpenCV 2 configuration used during testing. When using OpenCV 4, the corresponding include and library paths must be adapted to match the directory structure of the installed version):
 - Reference to the 'opencv/build/include' folder into the '[includePath](#)' list in the '[c_cpp_properties.json](#)' file.

- Manual LIBPATH reference to the 'opencv_world4110.lib' file in the task 'Linker' in the '**tasks.json**' file [see relative filepath in the sample].
4. Ensure that the VSCode terminal can use both the NVCC compiler (check with 'nvcc --version') and the MSVC compiler (check with 'cl'). In a manual VSCode configuration like this one, the best solution is to have both the 'cl' and 'nvcc' command references directly available in the IDE's terminal so that they can be configured in the 'tasks.json' file. In case the environment is having problems with the recognition of the commands 'cl' and 'nvcc', it is possible to consult the following sub-paragraphs.

MSVC (cl.exe) not recognised by the VSCode terminal:

The MSVC compiler, assuming to already have a correctly installed development environment, should be available from the terminal used by VS Code.

When running a project (either by 'F5' or by pressing the 'run' button) VSCode should ask the user which compiler shall be used if no one has been selected, proposing the cl compiler if detected.

If this tweak didn't force VSCode to recognise the 'cl' command, check the MSVC installation on the 'Visual Studio Installer' application: select the "C++ Compilation Tools" package and ensure to have the correct version of the MSVC compiler selected.

NVCC (nvcc.exe) not recognised by the VSCode terminal:

Not being a compiler supported by the Visual Studio installer, the NVCC compiler requires more attention to make so that the IDE may recognise it.

As first thing it is necessary that the device ambient variable 'CUDA_PATH' is correctly set; the installer is supposed to automatically create it during the installation (usually setting it as {Program Files}\NVIDIA GPU Computing Toolkit\CUDA\<version>).

If the cuda path is correctly set and linked to the VSCode environment, it is possible that residual data from past configurations may be creating conflicts with the current one.

To ensure that no past configuration is preventing the current one to be used it is possible to try the following solutions in sequence:

1. Soft cache deletion:

Delete the cache of any configuration tool installed in your VSCode (for instance CMake can dictate many configurations by info saved in its cache, so deleting it frequently is highly recommended when editing config files)

2. Drastic cache deletion:

If VS Code continues to use an outdated environment, clearing or renaming the relevant workspace storage may help force the application to recreate its cached configuration. So execute those two steps in order:

- Ensure that the '*Developer Command Prompt for VS <relase_year>*' can recognise the 'nvcc' command.
- Remove (or rename) the '{APPDATA}\Code\User\workspaceStorage' folder, then close every instance of VSCode and reopen it from the developer terminal. When VS Code is launched from the Developer Command Prompt, its integrated terminal can inherit the environment variables of the parent process.

2.2.2- Versioning

In this paragraph are listed the version of the used libraries and technologies.

- The recommended version for the OpenCV library, used for the user interface, is the version '4' (having the folder 'opencv\build\include\opencv4'), but the code written so far has also shown compatibility with version '2'.
- CUDA Toolkit 13.0 was initially considered; however, it was necessary to downgrade to CUDA Toolkit 11.3 to maintain compatibility with the NVIDIA driver supported by the development device, which is the '511.69' instead of the required '596.86'.
- Since CUDA Toolkit 11.3 supports only specific versions of the MSVC toolchain, the Visual C++ compiler also had to be selected accordingly. Therefore, an MSVC toolset compatible with CUDA 11.3 was used. For this reason the version selected for MSVC is the '142' (also referred as '14.20.27508' in the download website) and the version '19.20.27508.1' for 'cl.exe'.

3- CPU Implementation(C++)

The CPU component of the Parallel project is developed in C++ and is responsible for managing the image processing workflow and coordinating communication between the user interface, data management, image loading and saving, and the GPU module.

The project is organized into distinct modules. The CPU component uses the data structures defined in the **data_models** module (graphics.h and vector_filter.constants.h), while **image_loader** and **process_manager** are the main modules of the CPU component.

The project uses the traditional separation between .h header files and .cpp implementation files.

The **.h files** declare the functions and structures that must be made available to the other modules.

This separation helps keep the project organized and allows each module to be used by the others without needing to know the details of its implementation.

3.1 User Interface

The UI module serves as the point of interaction between the user and the program. In this project, **OpenCV** is used for image display and mouse event handling, while file selection dialogs are managed through the Windows APIs.

In this interaction with the user, the UI module interacting with the CPU module, creates the masks representing the image region selected by the user. Such service is implemented by the '**image_mask**' component, allowing the window opened by the ui to provide features based on mouse-events.

The function **mouse_callback()** implemented handles the events about cursor movement, and left button click and release.

This permits the interface to allow the user to click and drag onto the visualized image to draw a green square representing the selected area.

Creating the Mask

Once the selection is complete, the function **create_rectangle_mask()** is called.

Each position in the mask is then marked as selected, and the resulting mask is added to **std::vector<mask> masks**; this allows the user to create multiple masks, which are collected within the same vector.

Filter Selection

The UI also allows the user to choose the filter size(9 light, 13 medium, 21strong)

Output File Management

Before calling the Process Manager, the UI also constructs the output file path.

The path to the original image is processed using **split_with_divider()**, and a new filename is created so that the original image is not overwritten.

3.2 Data Model

The main data types used by the project are **pixel**, **vector_picture**, **mask** and **vector_filter**, which are defined in the **graphics.h** file.

These structures define the data format used for communication between the various components of the project.

pixel

The pixel structure represents a single pixel in the image; each pixel consists of three components (R, G, B), and the **unsigned char** data type is used for each channel.

vector_picture

The **vector_picture** structure represents an entire image, storing it as a vector of **struct 'pixels'** within the **data** array.

This representation allows the various modules of the project to use the same **vector_picture** structure and facilitates subsequent processing of the pixels by the GPU.

mask

The **mask** structure represents a region of the image to which the processing must be applied, having its own width, height and corner coordinates related to the input image.

A mask also has a **selection_vector**, which is a boolean sequence representing a selection of pixels within the mask's area to be selected for the operation.

The **selection_vector** is used to 'cut' the masks' region to avoid overlapping and to support irregularly-shaped selections.

In the project, masks are created through the user interface, which allows the user to select a rectangular region of the image using the mouse. The UI then constructs the corresponding mask object, which is subsequently passed to the Process Manager.

corner_coordinates, width, and height are also represented as unsigned short, since they are used to represent non-negative dimensions and coordinates and their values are not likely to reach for excessively high values.

vector_filter

The **vector_filter** structure, defined in the **graphics.h** file, is used to represent the 1-dimensional filters applied to blur the image.

Originally, the structure consisted of an array of coefficients, the array size, and the divisor to be applied to the sum as specified by the algorithm.

Later, during code development, the **divisor** value was removed.

This decision—since the divisor's value is 2 raised to the power of **size-1**—allowed for optimization of the calculation by replacing the division of two large numbers with a binary right shift.

The filters are stored in the file **vector_filter_constants.h** as pre-calculated values so that the coefficients that make them up can be accessed without having to calculate them at runtime.

Three filters are currently defined in the project (filter_9, filter_13, filter_21).

3.3 Process Manager

The Process Manager is the component that coordinates the process to execute the featured functionalities of the application; it does not directly handle user interaction nor contains any implementation to fulfill tasks.

Its main function is to receive data and option choices from the user and to call the required services, making an horizontal call to the image_loader to get image data and obtains the data of the requested pre-calculated filter from the set provided by the module 'src/data_models'.

Once the image, mask, and filter have been obtained, the Process Manager calls the 'blur_image()' function from the GPU access point provided by the 'image_blur_api.h' component which, being the host for the kernel call, acts as an intermediary between CPU and GPU.

The 'blur_image()' function returns a **vector_picture** representing the image resulting from the blurring operation.

Once the result is received from the GPU module, the Process Manager passes it to the Image Loader, which converts the **vector_picture** back into a **Mat** and saves the image to the specified path.

After calling the Process Manager, the UI uses output_image_path directly to open the resulting file and displays it in the "Result" window.

Image Loader

The Image Loader module handles image management at the file level and, using the OpenCV library, acts as a converter from the PNG image format to the `vector_picture` structure, which is then returned to the Process Manager.

4- GPU

The GPU module is composed of a set of dedicated sub-modules and relies on what are called “*kernel data models*”, specifically designed to ensure compatibility with the CUDA programming model and to efficiently manage the communication between the host and the GPU.

The implementation does not rely on external frameworks or third-party libraries for GPU programming. Instead, the module has been developed using only the essential CUDA extensions.

The following chapters describe the main components of the GPU implementation; covering algorithm description, data models usage, threads organization and grid topology.

4.1- Implemented Algorithm

In order to feature the blurring operation, the binomial algorithm has been chosen.

The algorithm consists in a series of calculations iterated through each involved pixel; within the selected area each pixel is updated through a weighted mean within the values of itself and the neighbouring pixels.

Theoretically speaking, in order to apply the calculation it is necessary to use a matrix filter; the filter is figuratively overlapped onto the pixel grid, positioned so that the iterated pixel to update is in the middle.

Since the values in the filter are distributed to have the bigger values in the middle, the central pixel is the one with the highest weight, while the neighboring pixels' influence grows weaker by the increasing distance from the centre.

Unfortunately this implementation has a complexity of $O(n^2)$, where ‘ n ’ is the length of the $n \times n$ filter.

This solution is not scalable, so a variant adopting a ‘separable filter’ is required. Instead of directly using a 2D filter, the same result can be obtained by applying two 1D filters sequentially, one for each dimension.

This approach reduces the complexity to $O(2n)$:

The first filter is applied horizontally to each row of pixels, producing an intermediate matrix of partial results. Each element of this matrix represents the weighted mean of a horizontal neighbourhood. The second filter is then applied vertically to the intermediate matrix. Therefore, each final value is calculated from vertical neighbours whose values already include the contribution of their respective horizontal neighbours.

As a consequence, although each individual filtering operation only involves a one-dimensional line of pixels, the combination of the two operations extends the influence over the entire two-dimensional neighbourhood. The corner pixels are therefore not neglected: their contribution is included in the intermediate results generated by the first filtering pass and is subsequently propagated to the final result by the second pass.

4.2- Kernel Data Models

The GPU module requires dedicated data models to represent and organize data according to the requirements of CUDA execution and memory management. These models are primarily used to adapt data structures originally designed for the CPU to a representation that can be efficiently transferred to and accessed by GPU kernels.

The most representative example of such a GPU-oriented data model is the **Device_mask_collection** struct. This structure was designed to allow efficient iteration over an array of Mask instances.

A **Mask** is a complex structure containing both primitive values and pointers to dynamically allocated data, as described in the CPU section.

Passing an array of **Mask** structures directly to a CUDA kernel introduces several issues. The most relevant are:

- An arbitrary number of masks would lead to an arbitrary number of allocations to perform, leading to a not scalable and hardly maintainable implementation.
- The data is provided as a set of non-continuous memory allocations, involving multiple levels of pointers. This situation compromises both efficiency and manageability of the accesses.

To address these issues, the **Mask array** is flattened into a single **Device_mask_collection** instance. The flattening process reorganizes the required data into contiguous arrays, allowing the GPU kernel to access and iterate over the data using the offsets stored within the resulting structure.

4.3- Threads Organization

At the base of the parallelization of operations on grids of data as images there's the subdivision of atomic tasks for each thread.

Such separation is possible because each task consists in a set of operations between values read on the selected and neighbouring pixels.

In order to guarantee the execution to have no conflicts **an output image has been allocated** as a distinct variable. While having two distinct allocations, one for input and the other for output, requires doubling the space occupied by the input image, this setup permits to prevent read-write conflicts by guaranteeing to have a read-only input.

An additional safeness is obtained by **cutting off any overlapping mask regions**, checking colliding coordinates and removing redundancies before the kernel call. This operation prevents write-write conflicts by ensuring that no writing operation is ever performed twice in the same allocated data before a synchronization.

The following chapter will explain in more detail how such configuration has been achieved:

4.3.1- Kernel Call

The blur kernel is launched using a three-dimensional grid composed of two-dimensional thread blocks.

This organization was chosen to efficiently map the selected regions defined by the masks. Since each mask represents an arbitrary rectangular selection, its dimensions may vary independently from those of the other masks.

To achieve an efficient distribution of the workload, a fixed block size of 16×16 threads was selected. This configuration provides a regular subdivision of the image plane and allows the available processing resources to be efficiently utilized.

The dimensions of each mask may vary independently. Consequently, the number of blocks required to cover a mask depends on its width and height, while the dimensions of individual blocks remain fixed. For each dimension, the required number of blocks is obtained by dividing the corresponding mask dimension by the block dimension and rounding the result up. This configuration ensures that the entire selected area is covered even when its dimensions are not exact multiples of the block size.

The third dimension of the grid is used to process multiple masks within the same kernel invocation. Each value of the z coordinate identifies a different mask and therefore an independent instance of the blur operation. This allows the workload associated with multiple image regions to be distributed simultaneously without requiring a separate kernel launch for each mask.

This approach is a trade-off applied to avoid having a kernel call for each mask at the cost of not providing the most fit grid dimensions for each mask. Indeed `gridDim.x` and `gridDim.y` must be large enough to cover the widest and tallest masks in the set.

Consequently, masks smaller than the maximum dimensions may be assigned blocks that correspond to regions outside their actual boundaries.

For this reason, additional bounds checks are required inside the kernel to prevent out-of-bounds memory accesses.

Every additional thread is discarded without performing useful computation, verifying if its computed coordinates belong to the mask's selected region or if that coordinate within the mask is selected for blurring.

In summary, following there's the logical correlations for the grid dimensions:

- **`threadIdx.x` and `threadIdx.y`**: Identify the relative coordinates of a thread within its block. Combined with the block coordinates, they determine the position of the pixel relative to the processed mask.
- **`blockIdx.x` and `blockIdx.y`**: Identify the block within the two-dimensional subdivision of the mask and contribute to determining the relative pixel coordinates.
- **`blockDim.x` and `blockDim.y`**: Fixed dimension to represent a regular portion in the cartesian plane of the image.
- **`blockIdx.z`**: Directly identifies the mask assigned to the current execution layer.
- **`gridDim.z`**: The number of masks processed by the kernel.
- **`gridDim.x` and `gridDim.y`**: Number of blocks required to cover the maximum width and maximum height among all masks in the set.

5- Conclusions and Results

The developed project can be considered satisfactory with respect to both its functional requirements and its parallel implementation.

The binomial blur algorithm represents a suitable problem for GPU acceleration, as the computation of the output pixels can be distributed among a large number of CUDA threads. Each block of threads is responsible for an independent portion of the output image, allowing the computational workload to be processed concurrently while avoiding concurrent writes to the same memory location.

Communication between the high-level OpenCV interface and the CUDA kernel module is simplified through the use of common data models, reducing the need for intermediate conversions. A representative example is the *`vector_image`* structure, which can be returned by the kernel computation and directly used by higher-level components without adaptations.

Furthermore the layer-based structure of the project permits, with the due adjustments on the 'ui.cpp' file, to be extended to support more blurring options or even other features entirely by adding more API accesses to the 'process_manager' component, which is responsible for preparing the internal data and executing the selected processing pipeline.

Overall, the project demonstrates how a computationally intensive image-processing operation can be implemented while also integrating the CUDA computation into a modular and scalable software architecture.

5.1- Possible Improvements

Although the current implementation fulfills the objectives of the project, several aspects could be further improved:

- During the development of the CUDA code, the absence of a debugger made the identification of failure points particularly challenging. For this reason, the horizontal and vertical convolution passes were implemented as two separate, almost identical functions, allowing failures to be isolated more easily.

The two functions are meant to be two separated kernel calls, having a synchronization in the between, so they would be merged into a single kernel function required to be called twice to perform both kinds of convolutions.

- Due to memory space limitations given by the 'unsigned int' variable type, it was possible to only implement a vectorial filter with 21 elements without exceeding the maximal value during the summing operations.

Since the goal of this project was just integration and optimization of GPU processes, the management of vastly large numbers has not been performed.

Future implementations could address this limitation through two different approaches: arbitrary-precision arithmetic could be introduced to represent larger values, or by the convolution formula rearrangement to apply the division during intermediate calculations, preventing the accumulated values from exceeding the limits of the current numerical representation.

- In the current state the project's applicative only permits to create rectangular selections during the masks creation, but the CUDA code actually supports irregularly-shaped masks and even performs partial or complete deselection of overlapping mask regions to avoid write-write conflicts.

This state of the project permits to implement more shapes for area selection by just updating the UI module; no cascade modification is foreseen as long as the defined data models stay unchanged.

These possible developments indicate that the current limitations are primarily localized and could be addressed without requiring substantial changes to the overall architecture of the project.